



PROGRAMAÇÃO WEB 2 · AULA 05

Navegando entre páginas

Hoje o nosso login vai levar a algum lugar...

Emanuel Nascente · Escola Técnica Mesquita

REACT :D



0 que vamos construir hoje

01

Ligar o **React Router** no projeto da aula 4 e desenhar o mapa das telas.

02

Fazer o login **levar** para uma segunda página — e a segunda página ter volta.

03

Trancar essa página: **sem login, ninguém entra** — e o Sair passa a sair de verdade.

04

Cuidar dos endereços que não existem e entender **por que nada disso é segurança**.

Preparando o terreno

01

Instalando

Na aula passada montamos a tela de login. Vamos continuar naquele mesmo projeto: abra a pasta dele no terminal e instale a biblioteca de rotas.

```
$ npm install react-router-dom
```

Depois de instalar, rode npm run dev de novo.

02

Criando

Quatro arquivos novos vão nascer dentro da pasta src ao longo da aula: a segunda página, o estilo dela, o guardião e a página de erro.

```
$ Home.jsx  
$ Home.css  
$ RotaPrivada.jsx  
$ NaoEncontrada.jsx
```

03

Mexendo

E três arquivos que já existem vão mudar: o main.jsx liga o Router, o App.jsx vira o mapa das telas e o Login.jsx aprende a navegar.

```
$ main.jsx  
$ App.jsx  
$ Login.jsx
```

Uma página só, várias telas

A página **nunca recarrega** — o React troca o que aparece na tela.

Uma aplicação React é uma SPA (Single Page Application): existe um único `index.html`, e é o React que decide qual componente desenhar dentro dele.

O React Router é quem faz essa decisão olhar para a barra de endereços. Cada endereço vira uma tela, e mudar de tela não custa um F5: nada pisca, nada é baixado de novo.

URL

`/home`

+

Route

`path="/home"`

=

Componente

`<Home />`

Ligar o Router

Dois arquivos que já existem. Nenhuma tela muda ainda: isto é encanamento, e sem ele nada do resto funciona.

A Abra `src/main.jsx` e importe o `BrowserRouter` do react-router-dom.

B Envolver o `<App />` com ele, dentro do render.

C Abra `src/App.jsx` e troque o `<Login />` por um `<Routes>` com duas rotas dentro.

import

```
main.jsx
1  import React from "react";
2  import ReactDOM from "react-dom/client";
3  import { BrowserRouter } from "react-router-dom";
4  import App from "./App";
5  import "./index.css";
6
7  ...
```

Tudo começa importando o que acabamos de instalar. Repare que o nome entre chaves é `BrowserRouter`, mas o pacote se chama `react-router-dom`.

Por que **Browser**Router? Porque ele usa o recurso de histórico do navegador — o mesmo que faz as setas de voltar e avançar funcionarem.

É por isso que, depois de logar, o botão de voltar do navegador leva de volta para o login. Não precisamos programar nada para isso.

BrowserRouter

```
main.jsx
1    ...
2
3    ReactDOM.createRoot(
4      document.getElementById("root")
5    ).render(
6      <React.StrictMode>
7        <BrowserRouter>
8          <App />
9        </BrowserRouter>
10     </React.StrictMode>
11   );
```

O BrowserRouter é um componente que ABRAÇA a aplicação inteira. Ele não desenha nada na tela: só fica de olho na barra de endereços.

Regra de ouro: tudo que usa rota precisa estar **dentro** do BrowserRouter. Por isso ele vem aqui no main.jsx, envolvendo o `<App />`.

Esquecer esse abraço é o erro mais comum da aula: o React acusa que os componentes de rota estão fora de um Router.

Routes

```
App.jsx

1  import { Routes, Route } from "react-router-dom";
2  import Login from "./Login";
3  import Home from "./Home";
4
5  function App() {
6    return (
7      <Routes>
8        <Route path="/" element={<Login />} />
9        <Route path="/home" element={<Home />} />
10     </Routes>
11   );
12 }
13
14 export default App;
```

Na aula passada o App.jsx só chamava o <Login />. Agora ele vira o mapa da nossa aplicação.

O **<Routes>** é a caixa que guarda todas as telas possíveis. Ele olha a URL atual, escolhe UMA das rotas de dentro dele e desenha só ela.

Uma de cada vez — nunca duas ao mesmo tempo.

Route

```
App.jsx

1  import { Routes, Route } from "react-router-dom";
2  import Login from "./Login";
3  import Home from "./Home";
4
5  function App() {
6    return (
7      <Routes>
8        <Route path="/" element={<Login />} />
9        <Route path="/home" element={<Home />} />
10     </Routes>
11   );
12 }
13
14 export default App;
```

Cada Route é uma linha do mapa, e ela sempre tem duas partes.

path é o endereço que aparece na barra do navegador. **element** é o componente que deve aparecer quando a URL for aquela.

O path "/" é a raiz: é o que abre quando você entra no site sem digitar mais nada. Por isso o login mora nele.

Repare na barra: **"/home"** e não **"home"**. Sem a barra vira caminho relativo.

Navegar entre as telas

Agora o projeto ganha a segunda página e o caminho de ida e volta até ela.

A Crie `src/Home.jsx` e `src/Home.css` — a página de destino e o estilo dela.

B No `Login.jsx`, chame o `useNavigate` e troque o `alert` por `navigate("/home")`.

C Na Home, um `<Link to="/">` para voltar. Teste entrando, voltando e usando as setas do navegador.

useNavigate

```
Login.jsx

1  import { useState } from "react";
2  import { useNavigate } from "react-router-dom";
3  import "./Login.css";
4
5  function Login() {
6    const [email, setEmail] = useState("");
7    const [senha, setSenha] = useState("");
8    const [erro, setErro] = useState("");
9
10   // o navigate troca de pagina pelo codigo
11   const navigate = useNavigate();
12
13   ...
```

O Login.jsx continua igualzinho ao da aula passada: os mesmos `useState`, o mesmo `handleSubmit`, o mesmo botão de mostrar senha.

A única novidade é o **`useNavigate`**. Ele é um Hook, assim como o `useState` — e como todo Hook, precisa ser chamado no topo do componente, nunca dentro de um `if`.

Ele não navega sozinho: devolve uma função (a que chamamos de `navigate`) que usamos quando quisermos trocar de página.

navigate("/home")

```
Login.jsx
1    ...
2
3    if (email === "user@email.com" &&
4        senha === "123456") {
5        setErro("");
6        navigate("/home");
7    } else {
8        setErro("E-mail ou senha incorretos.");
9    }
10   }
11
12   ...
```

Lembra desse if da aula passada? Ele é exatamente o mesmo. Só a linha do sucesso mudou.

Onde estava **alert("Login realizado...")** agora está o **navigate**. Em vez de uma caixinha do navegador, o usuário vai para outra tela.

O texto dentro do **navigate** é o mesmo path que escrevemos no Route lá no App.jsx. Se os dois não baterem, nada acontece.

navigate é para navegar por código, depois de uma decisão. Para navegar por clique existe o Link — próximo slide.

Home.jsx

```
Home.jsx

1  import { Link } from "react-router-dom";
2  import "./Home.css";
3
4  function Home() {
5    return (
6      <div className="container-home">
7        <div className="cartao">
8          <h2>Ponto Eletrônico</h2>
9          <p>Você entrou no sistema.</p>
10         <Link to="/">Sair</Link>
11       </div>
12     </div>
13   );
14 }
```

Esta é a nossa segunda tela. Repare que ela não tem nada de especial: é um componente comum, igual ao Login.

Não existe nenhum código dizendo "eu sou a página /home". Quem faz essa ligação é o Route lá no App.jsx — a página não precisa saber o endereço dela.

Por enquanto ela só dá as boas-vindas. Nas próximas aulas é aqui que vai nascer o registro de ponto.

export default Home;

Como sempre, essa linha no final é o que permite importar o componente no App.jsx.

Link

```
Home.jsx

1  import { Link } from "react-router-dom";
2  import "./Home.css";
3
4  function Home() {
5    return (
6      <div className="container-home">
7        <div className="cartao">
8          <h2>Ponto Eletrônico</h2>
9          <p>Você entrou no sistema.</p>
10         <Link to="/">Sair</Link>
11       </div>
12     </div>
13   );
14 }
```

O **<Link>** é o link do React Router. Ele vira um **<a>** no navegador, mas com um detalhe importante.

Um **<a href>** comum RECARREGA a página inteira: o React sobe do zero e todo o estado da aplicação se perde.

Já o Link troca a tela sem recarregar nada. E repare que ele usa **to**, não **href**.

Home.css

Home.css (1)

```
1  /* a Home tem o proprio container: ela nao
2     depende do Login.css para existir */
3
4  .container-home {
5      display: flex;
6      justify-content: center;
7      align-items: center;
8      height: 100vh;
9      background: #f0f2f5;
10 }
11
12 .cartao {
13     background: white;
14     padding: 2rem;
15     border-radius: 8px;
16     box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
17     width: 300px;
18     text-align: center;
19 }
```

Home.css (2) continuação

```
20
21 .cartao h2 {
22     margin-top: 0;
23 }
24
25 .cartao a {
26     display: inline-block;
27     margin-top: 1rem;
28     color: #4f46e5;
29     text-decoration: none;
30     font-weight: bold;
31 }
32
33 .cartao a:hover {
34     text-decoration: underline;
35 }
```

Trancar a porta

O coração da aula. Hoje a Home abre para qualquer um que digite o endereço; no fim desta etapa, não abre mais.

A No `Login.jsx`, grave a sessão com `localStorage.setItem` antes de navegar.

B Crie `src/RotaPrivada.jsx` e embrulhe a Home com ele no `App.jsx`.

C No `Home.jsx`, troque o Link de Sair por um botão que apaga a sessão — e ajuste o `Home.css`.

A porta está destrancada

Digite **/home** na barra de endereços sem fazer login. A página abre.

Isso não é um bug do React Router: é ele fazendo exatamente o que mandamos. O Route diz que a URL `/home` desenha a Home, e ninguém escreveu nenhuma condição além disso.

O mesmo vale para o Sair: ele leva de volta ao login, mas nada foi desligado — basta digitar `/home` outra vez para estar dentro.

Falta alguém entre a URL e a tela: um componente que pergunte “esta pessoa entrou?” antes de deixar a Home aparecer.

A URL

`/home`

+

O Route

`element={<Home />}`

=

O resultado

`entrou sem login`

Onde mora o “eu entrei”?

01

No useState?

Seria o caminho natural: um estado no App dizendo se a pessoa entrou. Só que estado de React vive na memória da página, e um F5 zera tudo.

```
$ useState(false)
```

Não serve: um F5 e o usuário está fora.

02

No localStorage

É uma gavetinha do próprio navegador. Sobrevive ao F5, a fechar a aba e a fechar o navegador. Guarda texto, e só texto. Três métodos dão conta de tudo.

```
$ setItem(chave, valor)
$ getItem(chave)
$ removeItem(chave)
```

Veja no DevTools: Application → Local Storage.

03

O que guardar

Por enquanto, uma marca simples: a chave logado com o texto “true”. Quando o back-end em FastAPI existir, aqui vai morar o token que ele devolve no login.

```
$ logado = "true"
```

setItem

```
Login.jsx

1    ...
2
3    if (email === "user@email.com" &&
4        senha === "123456") {
5        setErro("");
6        // marca a sessao como aberta
7        localStorage.setItem("logado", "true");
8        navigate("/home");
9    } else {
10       setErro("E-mail ou senha incorretos.");
11    }
12 }
13
14 ...
```

O if do login continua o mesmo. Antes de navegar, ele agora deixa um bilhete guardado no navegador: a sessão está aberta.

O **setItem** grava um par chave/valor. Os dois são texto — por isso **"true"** vai entre aspas, e não o booleano true.

O nome da chave é escolha nossa. O que não pode é escrever um nome aqui e outro na hora de conferir: são as duas pontas do mesmo combinado.

A ordem importa: **setItem** antes do **navigate**. Navegar primeiro é chegar na porta sem o bilhete na mão.

children

```
RotaPrivada.jsx

1  import { Navigate } from "react-router-dom";
2
3  function RotaPrivada({ children }) {
4    const logado =
5      localStorage.getItem("logado") === "true";
6
7    if (!logado) {
8      return <Navigate to="/" replace />;
9    }
10
11    return children;
12  }
13
14  export default RotaPrivada;
```

Este é o guardião, e ele cabe em um arquivo pequeno: um componente que não desenha nada de si mesmo. Ele só decide.

Tudo que você escreve **entre** as tags de um componente chega dentro dele na prop **children**. É a mesma ideia de uma `<div>` que embrulha outros elementos.

A conferência é uma linha só. Repare no **===** **"true"**: o `localStorage` devolve texto, nunca booleano.

getItem devolve **null** quando a chave não existe — e `null` não é `"true"`, então quem não entrou cai no `if`.

<Navigate />

RotaPrivada.jsx

```
1  import { Navigate } from "react-router-dom";
2
3  function RotaPrivada({ children }) {
4    const logado =
5      localStorage.getItem("logado") === "true";
6
7    if (!logado) {
8      return <Navigate to="/" replace />;
9    }
10
11    return children;
12  }
13
14  export default RotaPrivada;
```

Sem sessão, o guardião devolve um **<Navigate />** no lugar da tela. Renderizar esse componente É o redirecionamento.

Dois nomes parecidos, papéis diferentes: **navigate()** é a função do `useNavigate`, para usar dentro de uma ação; **<Navigate />** é o componente, para usar no meio do JSX — onde não dá para chamar função.

O **replace** substitui a entrada atual do histórico em vez de empilhar outra. Sem ele, o botão voltar devolveria a pessoa para /home e o guardião a expulsaria de novo: um pingue-pongue.

RotaPrivada

```
App.jsx
1    ...
2    import RotaPrivada from "../RotaPrivada";
3
4    function App() {
5      return (
6        <Routes>
7          <Route path="/" element={<Login />} />
8          <Route path="/home" element={
9            <RotaPrivada><Home /></RotaPrivada>
10          } />
11        </Routes>
12      );
13    }
```

Agora o guardião entra no mapa. O element da rota /home deixa de ser a Home solta e passa a ser a Home embrulhada.

Só o que está dentro do embrulho fica protegido. A rota "/" continua livre — e tem que continuar, senão ninguém chega nem no login.

Para proteger uma terceira tela amanhã, é a mesma linha: embrulha e pronto. O guardião não sabe — nem precisa saber — qual componente está protegendo.

Teste agora: **/home** na barra, sem login, tem que cair de volta na tela de entrada.

sair()

```
Home.jsx

1  import { useNavigate } from "react-router-dom";
2  ...
3  function Home() {
4    const navigate = useNavigate();
5
6    function sair() {
7      localStorage.removeItem("logado");
8      navigate("/");
9    }
10
11    ...
12    <button className="sair" onClick={sair}>
13      Sair
14    </button>
```

Com o guardião ligado, o **<Link>** de Sair virou uma mentira: ele volta para o login, mas a sessão continua aberta. Digite /home e você entra sem senha.

Sair é uma AÇÃO, não uma navegação: primeiro apaga o bilhete, depois navega. Por isso o Link vira um botão com onClick — e o useNavigate, que você já conhece do Login, reaparece aqui na Home.

O **removeItem** apaga a chave. Do lado de lá, o guardião volta a encontrar null e tranca a porta.

Cuidado com o nome da chave: **"logado"** aqui e no Login. Um erro de digitação e o Sair não sai.

.sair

```
Home.css

1  .cartao .sair {
2    margin-top: 1rem;
3    background: none;
4    border: none;
5    padding: 0;
6    font: inherit;
7    color: #4f46e5;
8    font-weight: bold;
9    cursor: pointer;
10 }
11
12 .cartao .sair:hover {
13   text-decoration: underline;
14 }
```

Trocar o link por um botão conserta o comportamento, mas estraga a aparência: o navegador desenha botão com fundo cinza, borda e fonte própria.

Estas quatro linhas são um **reset**: tiram o fundo, a borda e o respiro interno, e mandam o botão herdar a fonte do resto da página. O que sobra é um texto clicável.

O **cursor: pointer** devolve a mãozinha, que o link tem e o botão não.

Escreva no fim do Home.css, sem apagar nada do que já estava lá: as regras do .cartao continuam valendo.

Endereços que não existem

Duas pontas soltas antes de fechar a aula.

A No **App.jsx**, uma rota coringa **path="*"** para tudo que não bateu com as outras.

B Crie **src/NaoEncontrada.jsx**, a página 404, com um **<Link>** de volta para o login.

C Teste o furo que **sobra**: forje a chave no console do navegador e veja o guardião ceder.

path="*"

```
App.jsx
1  ...
2  import NaoEncontrada from "../NaoEncontrada";
3
4  function App() {
5    return (
6      <Routes>
7        <Route path="/" element={<Login />} />
8        <Route path="/home" element={
9          <RotaPrivada><Home /></RotaPrivada>
10       } />
11       <Route path="*"
12         element={<NaoEncontrada />} />
13     </Routes>
14    ...
```

E se alguém digitar **/hom**? Nenhuma rota bate, o Routes não desenha nada e a tela fica em branco — sem uma linha de erro no console.

O **path="*"** é o coringa: casa com qualquer endereço. Escrevemos por último para ler bem, mas a ordem não muda nada — o Router escolhe sempre a rota mais específica, não a primeira que encontra.

Repare que a rota coringa fica FORA do guardião: uma página de erro que exigisse login não faria sentido nenhum.

NaoEncontrada.jsx

```
NaoEncontrada.jsx

1  import { Link } from "react-router-dom";
2  // reaproveita o cartao da Home
3  import "./Home.css";
4
5  function NaoEncontrada() {
6    return (
7      <div className="container-home">
8        <div className="cartao">
9          <h2>404</h2>
10         <p>Esta página não existe.</p>
11         <Link to="/">Voltar ao login</Link>
12       </div>
13     </div>
14   );
```

A página de erro é o componente mais simples do projeto: um cartão, um aviso e o caminho de volta.

Ela importa o **Home.css** de propósito: as classes **.container-home** e **.cartao** já fazem exatamente o que precisamos. CSS se reaproveita entre telas.

E aqui o **<Link>** continua vivo: ir para o login é uma navegação comum, sem nada para desligar antes.

Não esqueça o **export default NaoEncontrada;** no fim do arquivo.

Isto não é segurança

O guardião protege a **tela** — nunca os **dados**.

Abra o Console do DevTools e digite `localStorage.setItem("logado", "true")`. Recarregue /home: a porta abre. Ninguém fez login.

A chave que o guardião confere mora no navegador do visitante — e o visitante manda no próprio navegador.

Guardião de rota é experiência do usuário. Segurança é o servidor recusar os dados — e é isso que vamos construir no back-end em Python com FastAPI.

A tela protegida

`RotaPrivada`

+

A marca no navegador

`localStorage`

=

Ainda assim

forjável em 10s

Testando o que construímos

01 Sem login, digite **/home** na barra: tem que cair na tela de entrada.

02 Entre com **user@email.com** e **123456**: a Home abre e a URL vira /home.

03 Aperte **F5** na Home: você continua dentro, porque a sessão está no localStorage.

04 Clique em **Sair** e tente /home de novo: tem que barrar. Se não barrar, a chave não foi apagada.

Os quatro erros de sempre

01

Erro `useNavigate()` may be used only in the context of a `<Router>`: faltou o `BrowserRouter` no `main.jsx`.

02

O `navigate` não leva a lugar nenhum: o texto dele e o `path` do `Route` têm que ser idênticos, barra inclusive.

03

Entra sem login mesmo com o guardião: a chave ficou salva de um teste anterior. Apague em **DevTools** → **Application**.

04

Tela branca ao abrir `/home`: o `RotaPrivada.jsx` precisa terminar com `return children;` — sem isso ele não desenha nada.

0 caminho inteiro, em quatro passos

01 Abraçar o App com `<BrowserRouter>` no main.jsx, depois do npm install.

02 Mapear as telas com `<Routes>` e `<Route>` no App.jsx, sem esquecer o coringa `path="*"`.

03 Navegar: `navigate()` por código, `<Link>` por clique.

04 Proteger: `<RotaPrivada>` embrulhando a rota, e o Sair apagando a sessão.

0 teu trabalho

- 01 Criar uma **terceira página** no projeto — o assunto dela é escolha tua.
- 02 Registrar a rota dela no App.jsx e pôr um `<Link>` na Home levando até ela — e outro de volta.
- 03 Proteger a rota nova com o `<RotaPrivada>`, do mesmo jeito que fizemos com a Home.
- 04 Testar o furo: digite o endereço da página nova **sem ter feito login**. Tem que cair na tela de entrada.